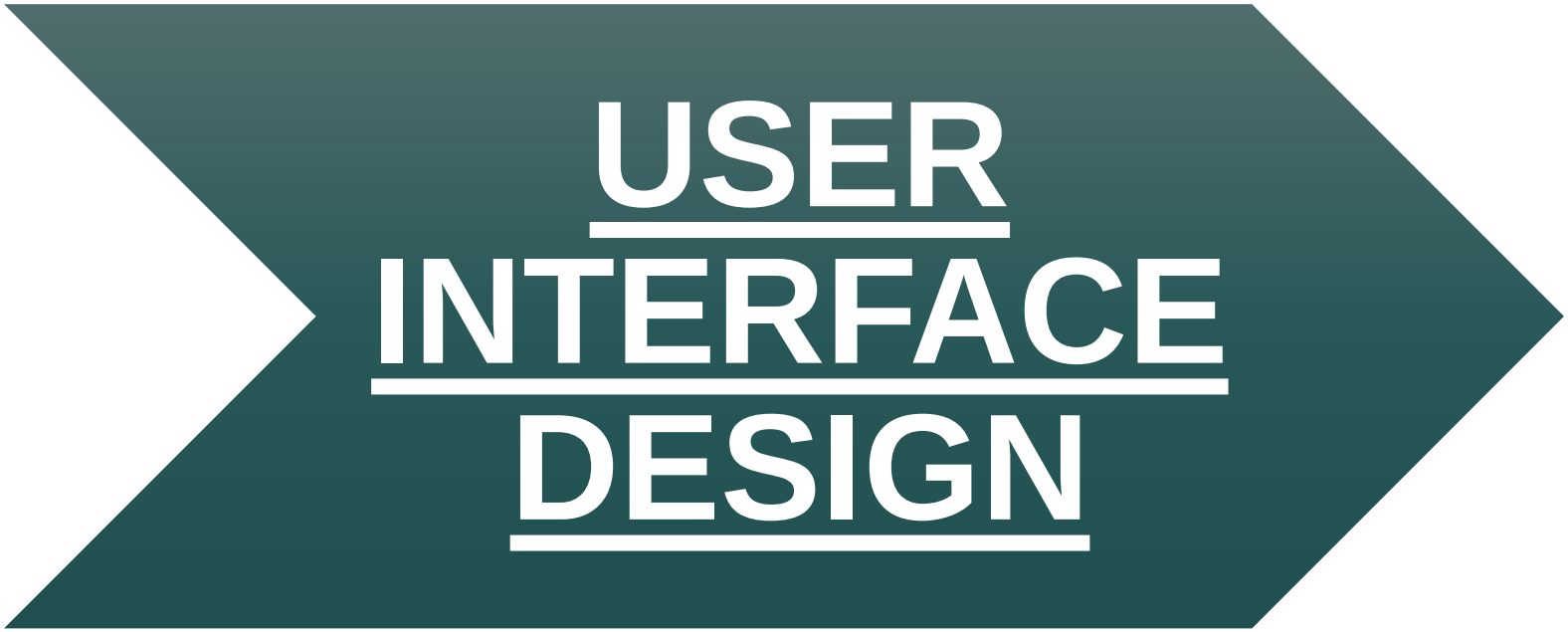
UNIT 3

1. User Interface Design
2. Unified Process Model
3. Coding Standards and Guidelines

Software engineering



Types of user interfaces

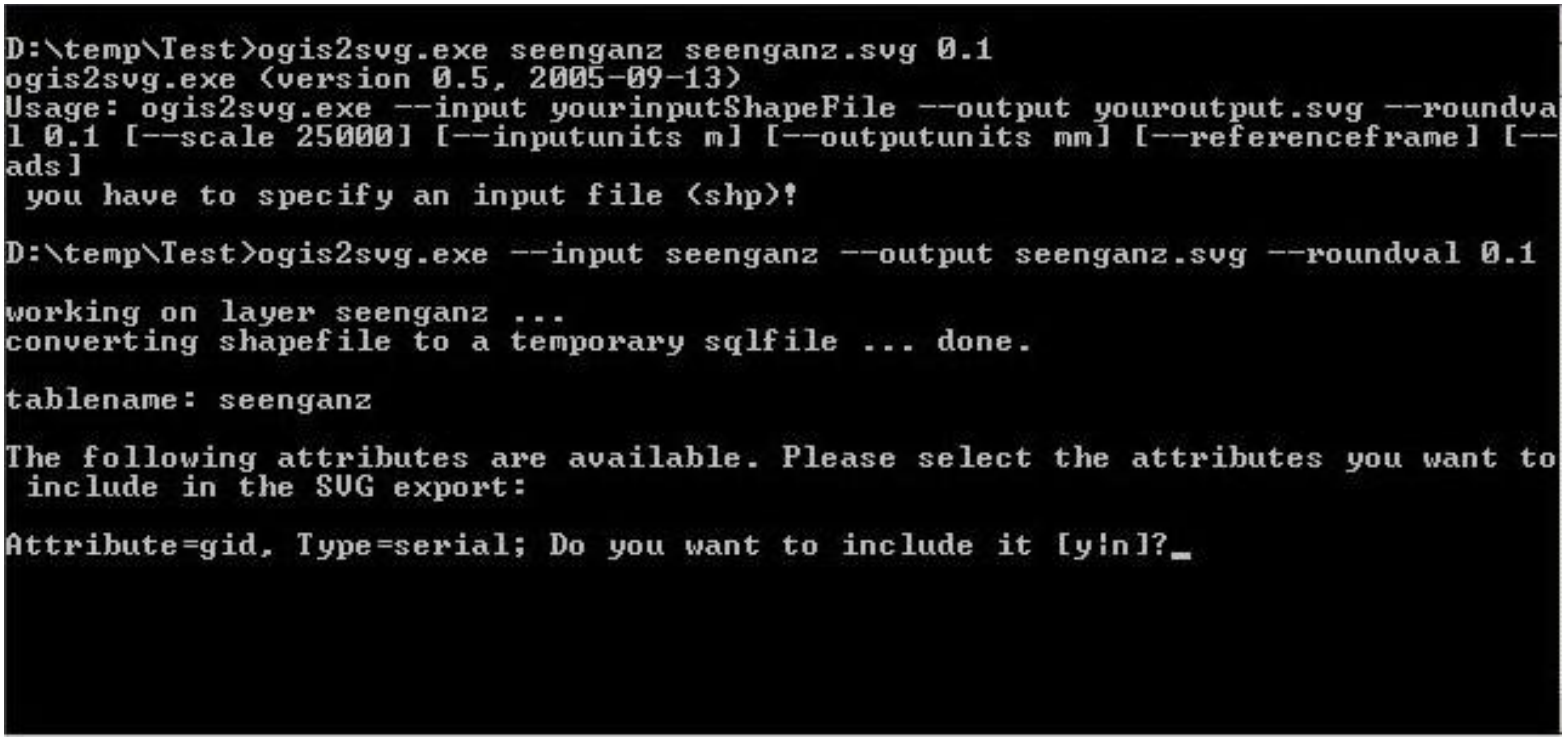
- User interfaces can be classified into the following three categories:
  1. Command language based interfaces
  2. Menu-based interfaces
  3. Direct manipulation interfaces (Iconic)


GUI
VS
COMMAND LINE
Start

Command Language-based Interface

- A command language-based interface – as the name itself suggests, is based on designing a command language which the user can use to issue the commands.
- The user is expected to frame the appropriate commands in the language and type them in appropriately whenever required.
- A command language-based interface can be made concise requiring minimal typing by the user.
- Command language-based interfaces allow fast interaction with the computer and simplify the input of complex commands.

Command Language-based Interface

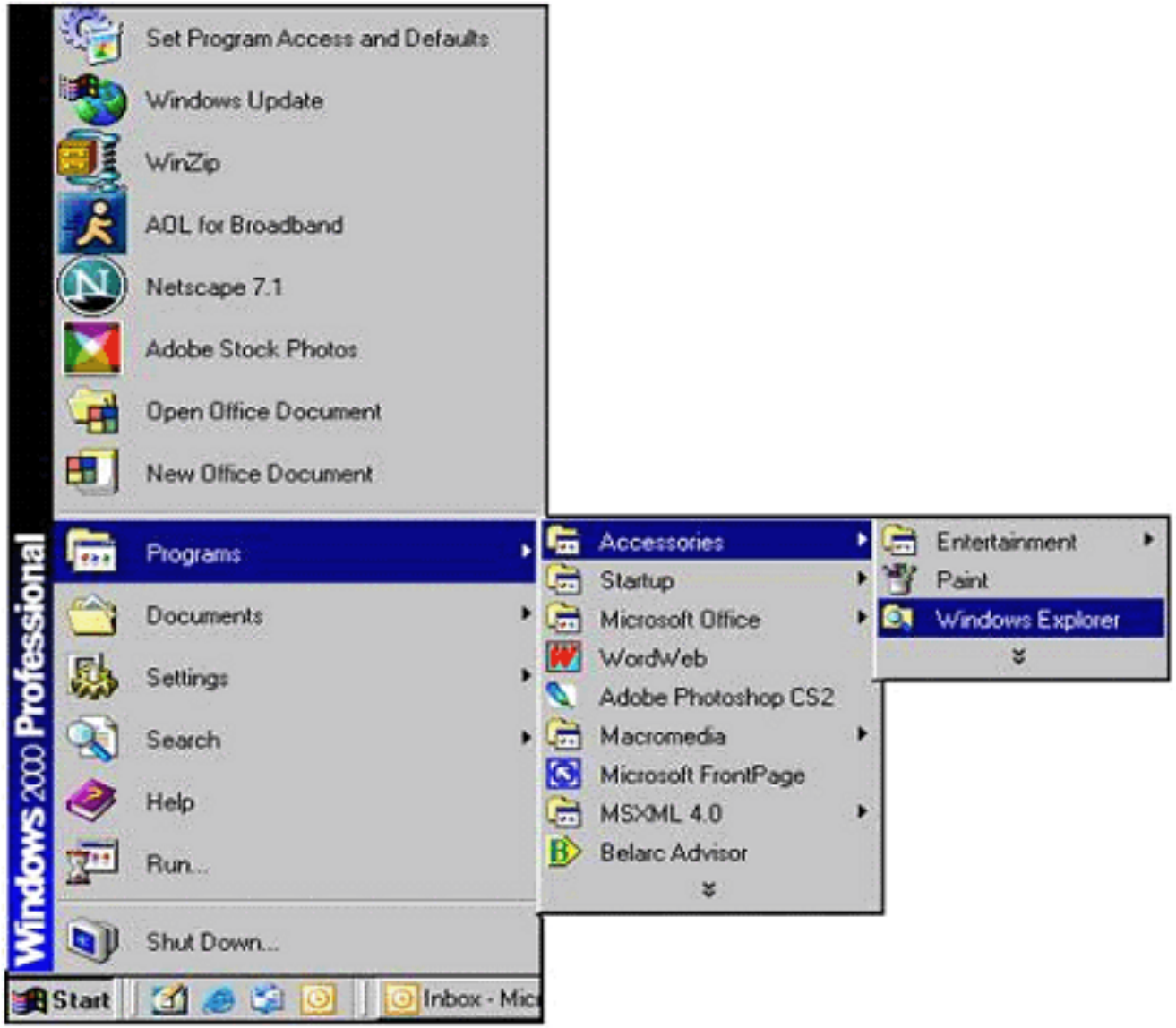


Menu-based Interface

- An important advantage of a menu-based interface over a command language-based interface is that a menu-based interface **does not require the users to remember the exact syntax of the commands**.

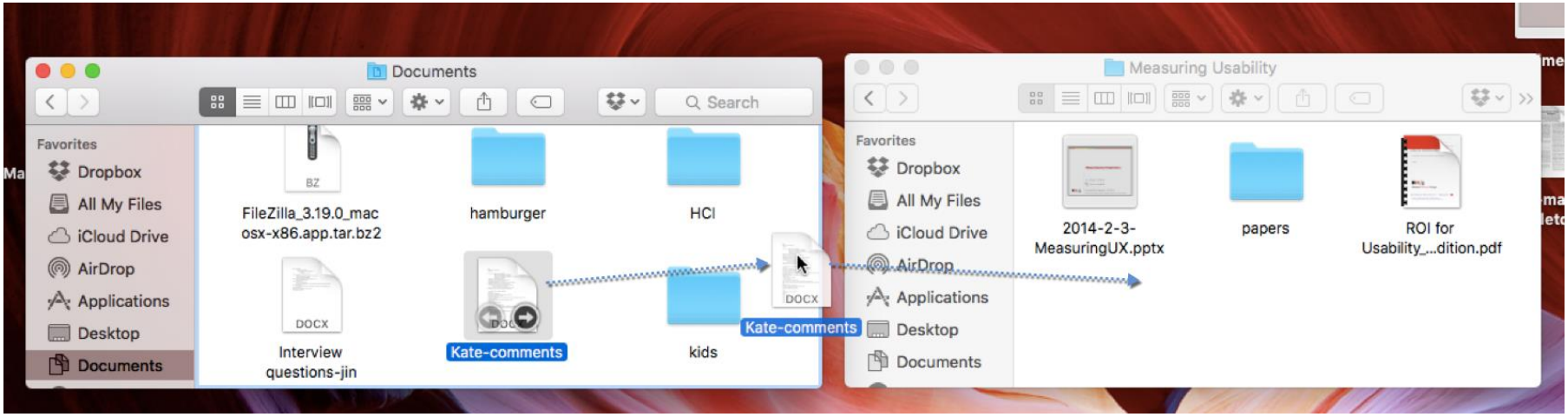
- A menu-based interface is based on **recognition of the command names, rather than recollection**.

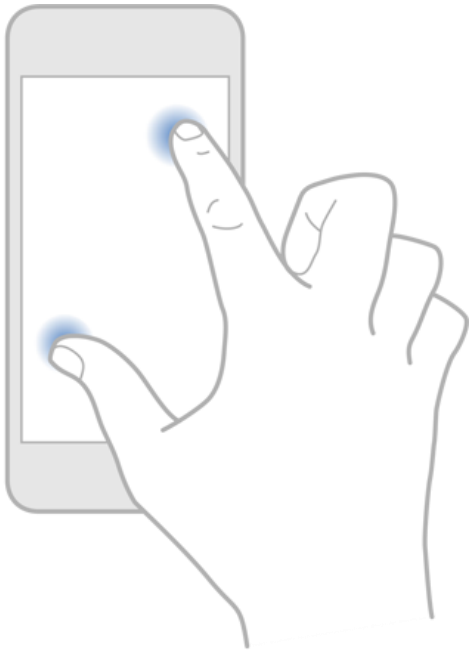
- Further, in a menu-based interface the **typing effort is minimal** as most interactions are carried out through menu selections using a pointing device.


Set Program Access and Defaults
Windows Update
WinZip
AOL for Broadband
Netscape 7.1
Adobe Stock Photos
Open Office Document
New Office Document
Programs
Documents
Settings
Search
Help
Run...
Shut Down...
Windows 2000 Professional
Start
Inbox - Mic
Accessories
Startup
Microsoft Office
WordWeb
Adobe Photoshop CS2
Macromedia
Microsoft FrontPage
MSXML 4.0
Belarc Advisor
Entertainment
Paint
Windows Explorer

Direct Manipulation Interfaces

- Direct manipulation interfaces present the interface to the user in the form of visual models (i.e. **icons or objects**).
- For this reason, direct manipulation interfaces are sometimes called as **iconic interface**.
- In this type of interface, the user issues commands by performing actions on the visual representations of the objects, e.g. **pull an icon representing a file into an icon representing a trash box, for deleting the file.**
- Important advantages of iconic interfaces include the fact that **the icons can be recognized by the users very easily, and that icons are language-independent.**


Documents
Search
Favorites
Dropbox
All My Files
iCloud Drive
AirDrop
Applications
Desktop
Documents
FileZilla_3.19.0_mac osx-x86.app.tar.bz2
hamburger
HCI
Interview questions-jin
Kate-comments
kids
Kate-comments
Measuring Usability
Favorites
Dropbox
All My Files
iCloud Drive
AirDrop
Applications
Desktop
Documents
2014-2-3-MeasuringUX.pptx
papers
ROI for Usability_...dition.pdf



Characteristics of a user interface

- **Speed of learning.** *A good user interface should be easy to learn*.
- **Speed of learning is hampered by complex syntax and semantics of the command issue procedures.**
- **A good user interface should not require its users to memorize commands.**

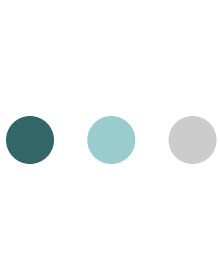
- **Speed of use.** *Speed of use of a user interface is determined by the time and user effort necessary to initiate and execute different commands*.
- **Speed of recall.** *Once users learn how to use an interface, the speed with which they can recall the command issue procedure should be maximized.*

- **Error prevention.** A good user interface should minimize the scope of committing errors while initiating different commands.

- **Attractiveness.** A good user interface should be attractive to use. In this respect, graphics-based user interfaces have a definite advantage over text-based interfaces.
- **Consistency.** The commands supported by a user interface should be consistent. Consistency facilitates speed of learning, speed of recall, and also helps in reduction of error rate.
- **Feedback.** A good user interface must provide feedback to various user actions.

- **Error recovery (undo facility).** While issuing commands, even the expert users can commit errors. Therefore, a good user interface should allow a user to undo a mistake committed by him while using the interface.
- **User guidance and on-line help.** Users seek guidance and on-line help when they either forget a command or are unaware of some features of the software. Whenever users need guidance or seek help from the system, they should be provided with the appropriate guidance and help.

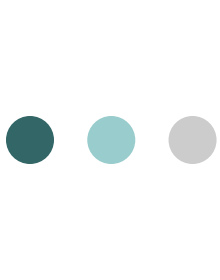
- A mode is a state or collection of states in which only a subset of all user interaction tasks can be performed.
- **In a modeless interface**, the same set of commands can be invoked at any time during the running of the software.
- Thus, a modeless interface has only a single mode and all the commands are available all the time during the operation of the software.
- On the other hand, in a **mode-based interface**, different set of commands can be invoked depending on the mode in which the system is.



- **Mode-based interface**

vs.

- **Modeless interface**



A mode is a state or collection of states:

- in each state (or mode) different types of commands are available.

Modeless interface:

- same set of commands are available at all times.

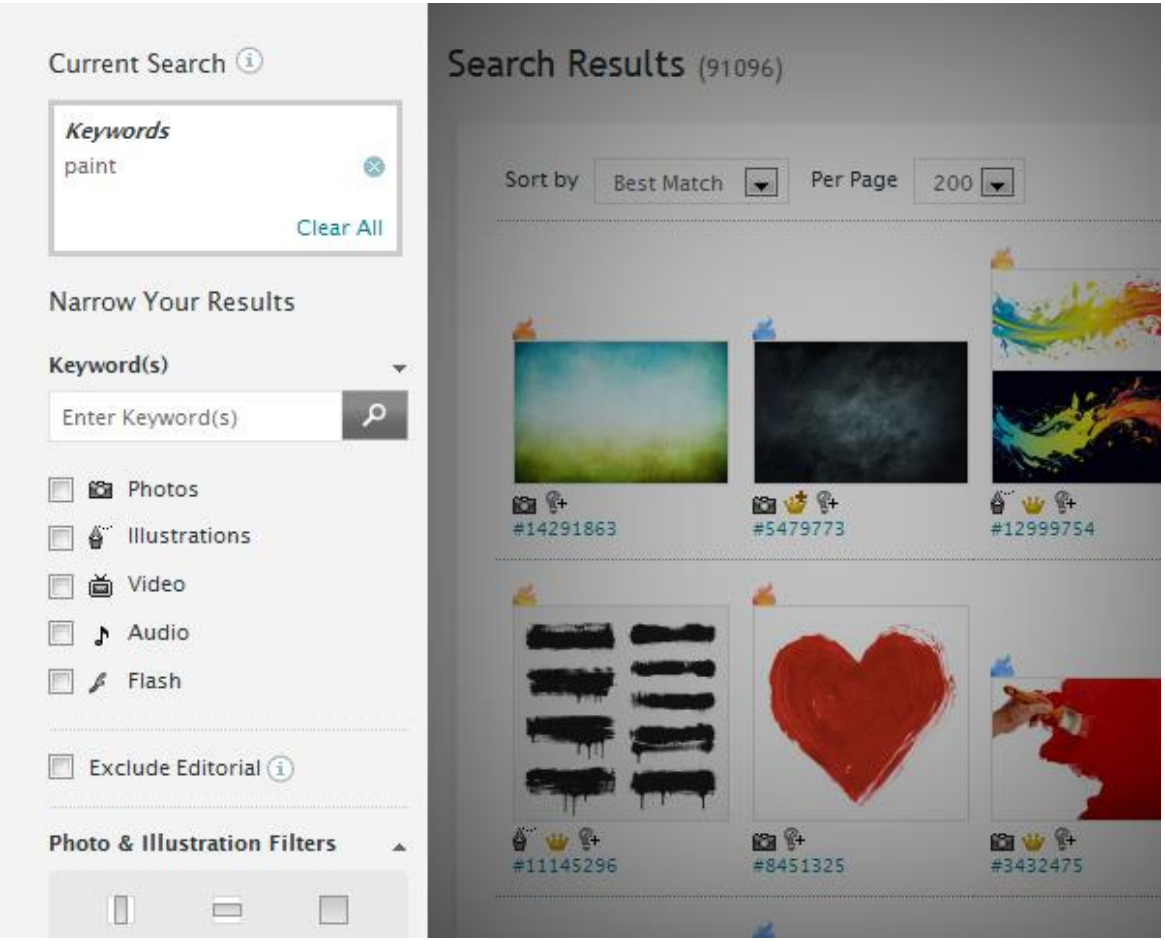
Mode-based interface:

- different sets of commands are available depending on the mode in which the system is, i.e.
- based on the past sequence of the user commands.

BASIC CONCEPTS:

Mode-based interface vs. modeless interface

Modeless:


Current Search
Keywords
paint
Clear All
Narrow Your Results
Keyword(s)
Enter Keyword(s)
Photos
Illustrations
Video
Audio
Flash
Exclude Editorial
Photo & Illustration Filters
Search Results (91096)
Sort by
Best Match
Per Page
200
#14291863
#5479773
#12999754
#11145296
#8451325
#3432475

- Graphical User Interface

vs.

- Text-based User Interface

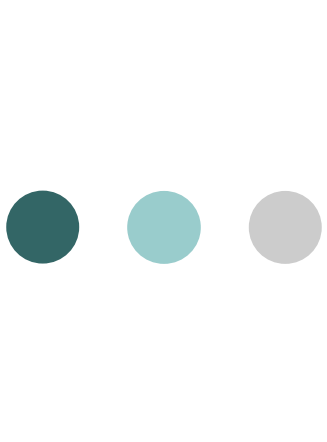
Graphical User Interface vs. Text-based User Interface

- **In a GUI multiple windows with different information can simultaneously be displayed on the user screen.**
- **This is perhaps one of the biggest advantages of GUI over text-based interfaces.**
- **Iconic information representation and symbolic information manipulation is possible in a GUI.**

- A GUI usually supports command selection using an attractive and user-friendly menu selection system.
- In a GUI, a pointing device such as a mouse or a light pen can be used for issuing commands. The use of a pointing device increases the efficacy issue procedure.

Some advantages about TBUI

- On the other hand, a text-based user interface can be implemented even on a cheap alphanumeric display terminal.
- Graphics terminals are usually much more expensive than alphanumeric terminals.



Unified Process Model

What is the Unified Process

A popular iterative modern process model (framework) derived from the work on the UML and associated process.

The leading object-oriented methodology for the development of large-scale software

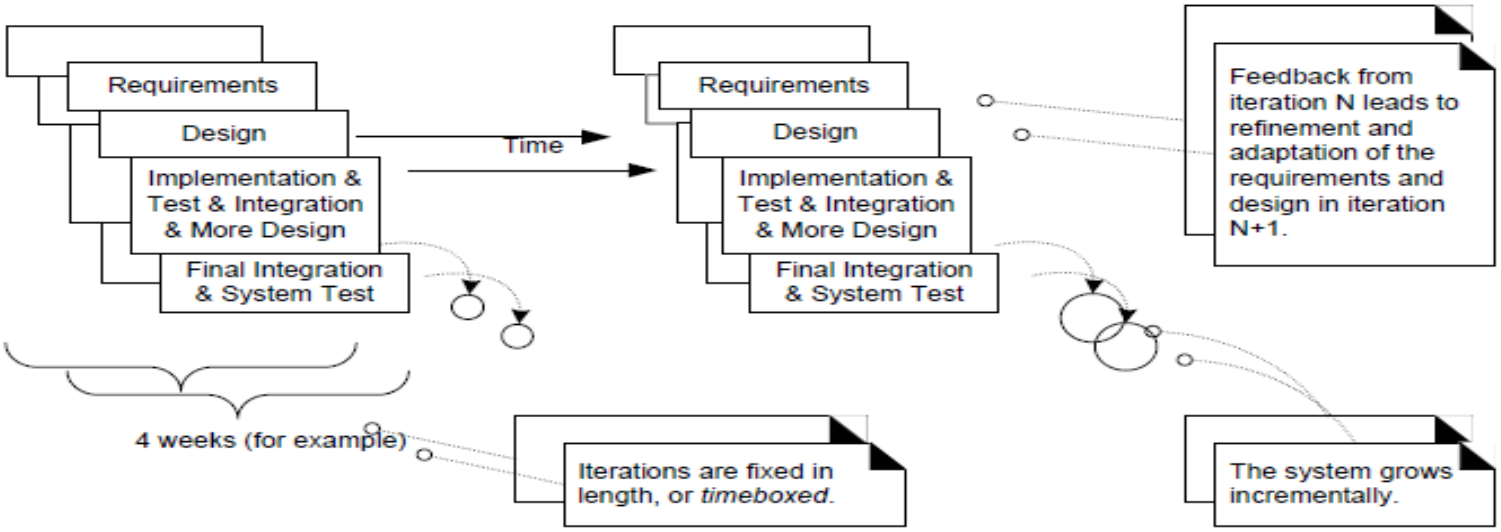
Maps out when and how to use the various UML techniques

The Most Important UP Idea: Iterative Development

- The UP(unified process) promotes several best practices, but one stands above the others: **iterative development.**
- In this approach, development is organized into a series of short, fixed-length (for example, four week) mini-projects called **iterations.**
- The outcome of each is a tested, integrated, into executable system.
- Each iteration includes its own requirements analysis, design, implementation, and testing activities.

Iterative and incremental development

- The system grows incrementally over time, iteration by iteration, and thus this approach is also known as **iterative and incremental development.**


Requirements
Design
Implementation & Test & Integration & More Design
Final Integration & System Test
Time
Requirements
Design
Implementation & Test & Integration & More Design
Final Integration & System Test
Feedback from iteration N leads to refinement and adaptation of the requirements and design in iteration N+1.
4 weeks (for example)
Iterations are fixed in length, or *timeboxed*.
The system grows incrementally.

Imp point about phases.

- This is *not* the old "waterfall" or sequential lifecycle of first defining all the requirements, and then doing all or most of the design.

- Inception is not a requirements phase; rather, it is a kind of feasibility phase, where just enough investigation is done to support a decision to continue or stop.

The UP Phases

A UP project organizes the work and iterations across four major phases:

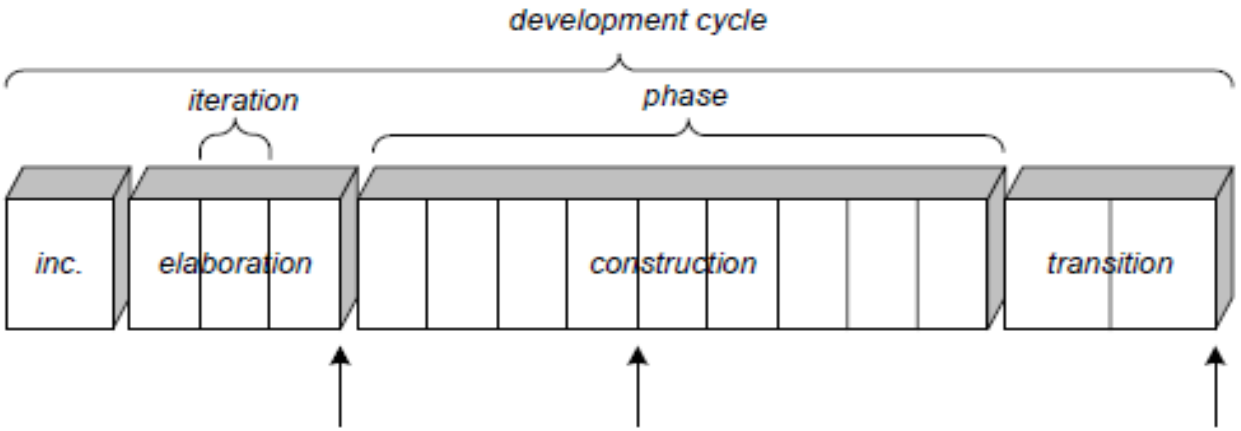
1.Inception— approximate vision, business case, scope, vague estimates.

2.Elaboration—refined vision, iterative implementation of the core architecture, resolution of high risks, identification of most requirements and scope, more realistic estimates.

3.Construction—iterative implementation of the remaining lower risk and easier elements, and preparation for deployment.

4.Transition—beta tests, deployment.

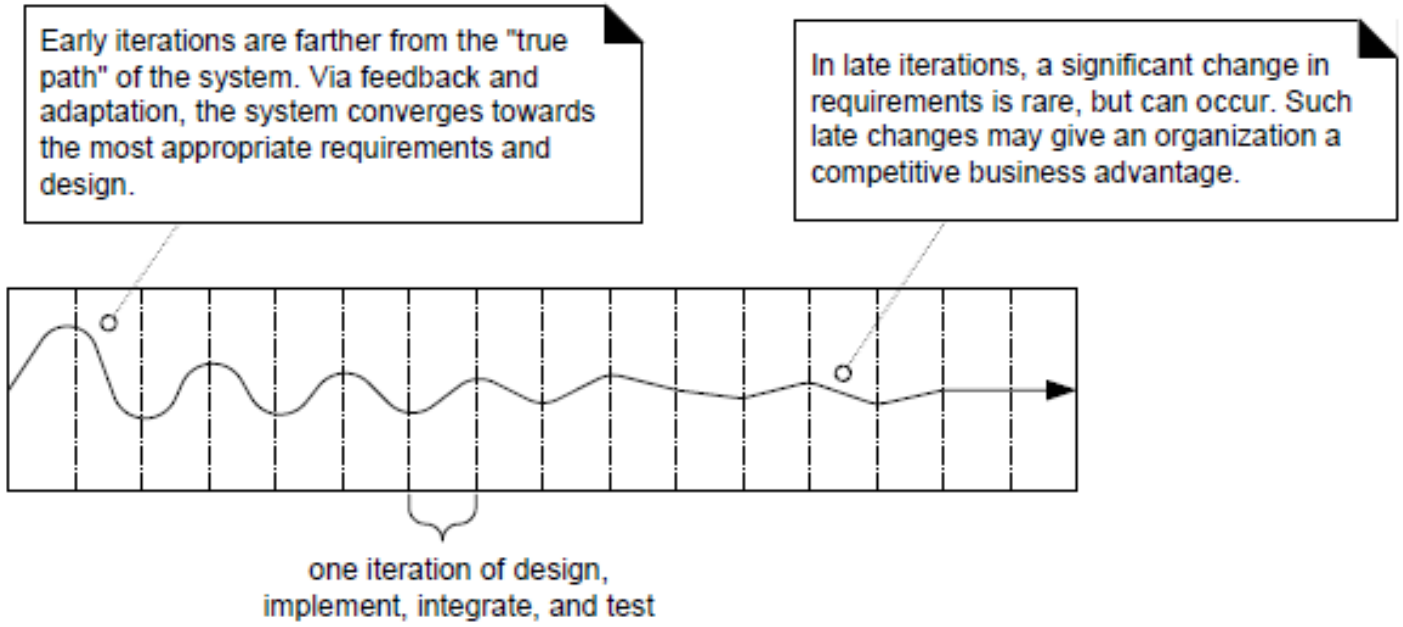
Schedule-oriented terms in the UP.



Iteration result..

- The result of each iteration is an executable but incomplete system.
- The output of an iteration is *not* an experimental or throw-away prototype, and iterative development is not prototyping. Rather, the output is a production-grade subset of the final system.

Iterative feedback and adaptation leads towards the desired system.



Benefits of Iterative Development

It includes:

- early rather than late mitigation of high risks (technical, requirements, objectives, usability, and so forth)
- early visible progress
- early feedback, user engagement, and adaptation, leading to a refined system that more closely meets the real needs of the stakeholders
- managed complexity; the team is not overwhelmed by "analysis paralysis" or very long and complex steps
- the learning within an iteration can be methodically used to improve the development process itself, iteration by iteration

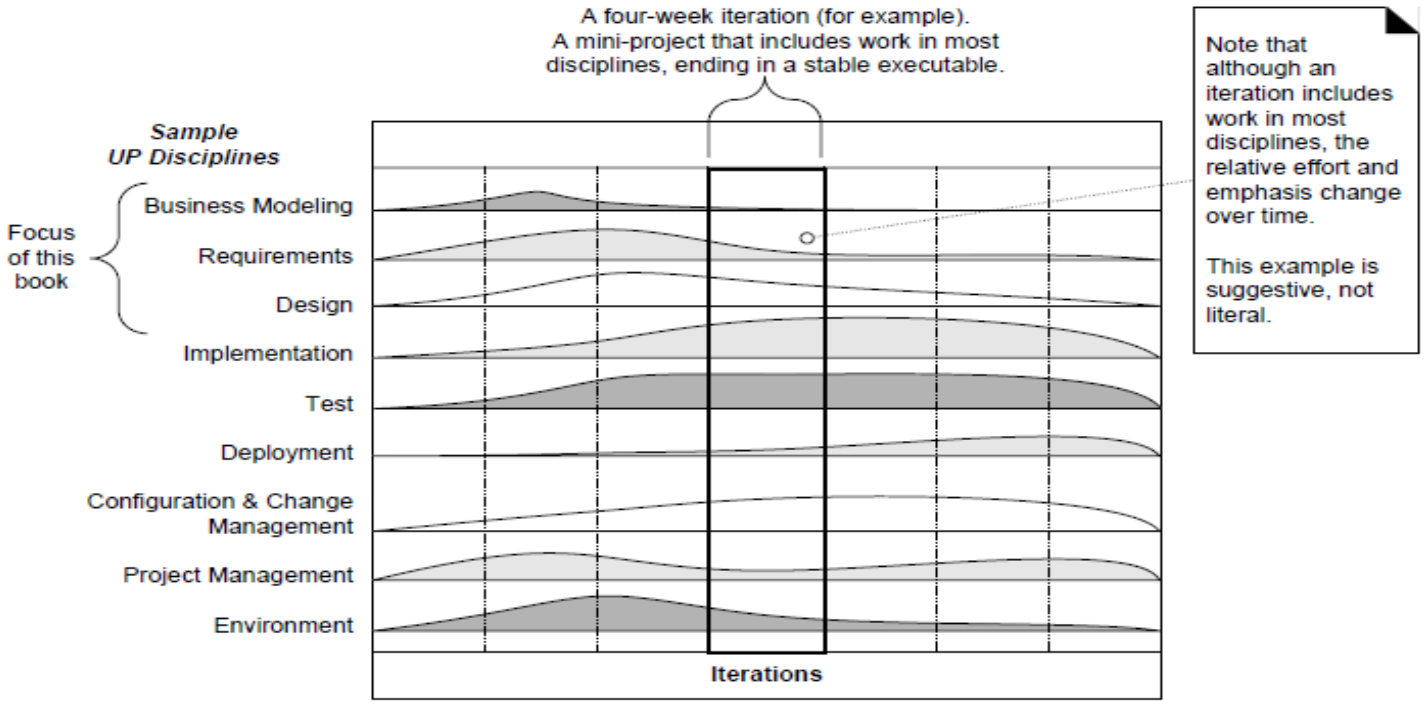
Iteration Length

- The UP (and experienced iterative developers) recommends an iteration length between two and six weeks.
- Much less than two weeks, and it is difficult to complete sufficient work to get meaningful throughput and feedback;
- Much more than six or eight weeks, and the complexity becomes rather overwhelming, and feedback is delayed.
- A very long iteration misses the point of iterative development.
- Short is good.

Time-boxed

- A key idea is that iterations are **time boxed,** or fixed in length.
- For example, if the next iteration is chosen to be four weeks long, then the partial system should be integrated, tested, and stabilized by the scheduled **date—date slippage is discouraged.**
- If it seems that it will be difficult to meet the deadline, the recommended response is to remove tasks or requirements from the iteration, and include them in a future iteration, rather than slip the completion date.

UP disciplines as Resource Histogram



Coding Standards and Guidelines

Advantages Coding Standards and Guidelines

1. coding standards give a uniform appearance to the codes written by different engineers

2. Increases code understandability and reuse

3. Promotes good programming practices

Coding Standards and Guidelines

1. Rules for limiting the use of global variables
2. Naming conventions for variables and constants identifiers
3. Conventions regarding error return values and exception handling mechanisms
4. Representative coding guidelines:

-- do not use a coding style that is too clever or difficult to understand

Coding Standards and Guidelines

--do not use an identifier for multiple purposes

--code should be well documented

-- length of any function should not exceed 10 source lines

--do not use GOTO statement

Code Review

- Types of review:

  -- code inspection

  -- code walkthrough (author presents code)